

Handwritten Digit Recognition using Convolutional Neural Network (CNN)

A Minor Project Report

Submitted in partial fulfillment of requirement of the
Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Mayuresh Khedkar

EN23CS301608

Under the Guidance of

Dr. Garima Silakari Tukra

Dr. Sheetal Bawane



Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE – 453331

APRIL – 2026

Report Approval

The project work "**Handwritten Digit Recognition using Convolutional Neural Network (CNN)**" is hereby approved as a creditable study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as a prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the Project Report only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation:

Affiliation:

External Examiner

Name:

Designation:

Affiliation:

Declaration

I hereby declare that the project entitled "**Handwritten Digit Recognition using Convolutional Neural Network (CNN)**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering, completed under the supervision of **Dr. Garima Silakari Tukra and Dr. Sheetal Bawane**, Faculty of Engineering, Medi-Caps University, Indore, is an authentic work.

Further, I declare that the content of this Project work, in full or in parts, has neither been taken from any other source nor has been submitted to any other Institute or University for the award of any degree or diploma.

Signature of the Student: _____

Name: Mayuresh Khedkar

Roll No.: EN23CS301608

Date: _____

Certificate

We, **Dr. Garima Silakari Tukra and Dr. Sheetal Bawane**, certify that the project entitled "**Handwritten Digit Recognition using Convolutional Neural Network (CNN)**", submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering by **Mayuresh Khedkar (EN23CS301608)**, is the record of work carried out by him under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Dr. Garima Silakari Tukra and
Dr. Sheetal Bawane
CSE DEPARTMENT
Medi-Caps University, Indore

External Guide.....
CSE DEPARTMENT
Medi-Caps University, Indore

Dr. Kailash Chandra Bandhu
Head of the Department
Computer Science & Engineering
Medi-Caps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R.C. Mittal**, who has provided every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D.K. Patnaik**, Vice Chancellor, Medi-Caps University, whose unfailing support has consistently boosted my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya**, Dean, Faculty of Engineering, Medi-Caps University, for providing the opportunity to work on this project, and **Dr. Kailash Chandra Bandhu**, Head of the Department, for his continuous encouragement.

I extend my sincere gratitude to my project guides, **Dr. Garima Silakari Tukra** and **Dr. Sheetal Bawane**, for their expert guidance, patience, and invaluable support throughout the development of this project. Their technical insights and constructive feedback were instrumental in shaping this work.

I am grateful for their help and support, which enabled me to complete this technical report.

Without their support this report would not have been possible.

Mayuresh Khedkar

B.Tech. III Year

Department of Computer Science & Engineering

Faculty of Engineering

Medi-Caps University, Indore

Abstract

Handwritten digit recognition is a fundamental problem in the field of computer vision with widespread real-world applications including postal automation, bank cheque processing, and digitisation of handwritten records. This project presents the design, development, and deployment of an automated Handwritten Digit Recognition System using a Convolutional Neural Network (CNN) trained on the MNIST dataset.

The system is implemented as a Flask-based web application, allowing users to upload an image of a handwritten digit and receive an instant prediction. The CNN model is built using TensorFlow and Keras, comprising two convolutional blocks with MaxPooling and Dropout regularisation, followed by fully connected dense layers with Softmax output. The model is trained on 63,000 samples from the MNIST dataset (90% training, 10% testing), with input images normalised to a 28×28 grayscale format.

Multiple training configurations were evaluated across epochs 8, 10, 11, 12, and 15. The model trained for 10 epochs was selected as the final deployment model, achieving a test accuracy of approximately 99.08% and a test loss of 0.028 on the MNIST test set. On real-world handwritten samples (30+ images), the system achieved approximately 80–82% accuracy, demonstrating strong generalisation.

The backend preprocessing pipeline handles grayscale conversion, resizing to 28×28 , colour inversion to match MNIST format, pixel normalisation, and array reshaping. The frontend is developed using HTML and CSS with a responsive, card-based layout. The project demonstrates end-to-end integration of deep learning, web development, and image processing for a practical recognition application.

Keywords: Convolutional Neural Network, MNIST, TensorFlow, Flask, Digit Recognition, Image Preprocessing, Deep Learning, Python

Table of Contents

	Section	Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgements	v
	Abstract	vi
	Table of Contents	vii
	List of Figures	ix
	List of Tables	x
	Abbreviations	xi
Chapter 1	Introduction	
	1.1 Introduction	1
	1.2 Literature Review	1
	1.3 Objectives	2
	1.4 Significance	2
	1.5 Research Design	3
	1.6 Source of Data	3
	1.7 Chapter Schema	3
Chapter 2	Requirements Specification	
	2.1 User Characteristics	4
	2.2 Functional Requirements	4
	2.3 Dependencies	5
	2.4 Performance Requirements	5

	2.5 Hardware Requirements	5
	2.6 Constraints & Assumptions	6
Chapter 3	Design	
	3.1 Algorithm	7
	3.2 System Design	9
	3.3 Database Design	16
Chapter 4	Implementation, Testing, and Maintenance	
	4.1 Languages, IDEs, Tools and Technologies	18
	4.2 Testing Techniques and Test Plans	19
	4.3 Installation Instructions	20
	4.4 End User Instructions	20
Chapter 5	Results and Discussions	
	5.1 User Interface Representation	21
	5.2 Brief Description of System Modules	22
	5.3 Snapshots with Brief Descriptions	23
Chapter 6	Summary and Conclusions	24
Chapter 7	Future Scope	25
	Appendix A: Glossary	26
	Bibliography	27

List of Figures

Figure No.	Figure Title	Page No.
Figure 3.1	CNN Architecture Block Diagram	7
Figure 3.2	Data Flow Diagram – Level 0 (Context Diagram)	9
Figure 3.3	Data Flow Diagram – Level 1	10
Figure 3.4	Activity Diagram – Digit Prediction Workflow	12
Figure 3.5	Sequence Diagram – Prediction Request Flow	14
Figure 3.6	Class Diagram – System Components	15
Figure 5.1	Home Page – Upload Interface	21
Figure 5.2	Prediction Result Display	22

List of Tables

Table No.	Table Title	Page No.
Table 2.1	Functional Requirements	4
Table 3.1	CNN Architecture Layers	7
Table 3.2	Data Dictionary	16
Table 4.1	Training Configurations and Results (Epoch-wise Model Performance)	19
Table 4.2	Test Cases and Results	19

Abbreviations

Abbreviation	Full Form
CNN	Convolutional Neural Network
MNIST	Modified National Institute of Standards and Technology
ReLU	Rectified Linear Unit
API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
PIL	Python Imaging Library
IDE	Integrated Development Environment
DL	Deep Learning
ML	Machine Learning
UI	User Interface
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
CORS	Cross-Origin Resource Sharing
WSGI	Web Server Gateway Interface

Chapter 1 - Introduction

1.1 Introduction

Handwritten digit recognition is a classical and well-studied problem in the field of computer vision and pattern recognition. The ability of a machine to accurately identify handwritten numerals from image data has substantial practical significance in domains such as bank cheque processing, automatic postal sorting, form digitisation, and automated data entry systems.

Traditional approaches to digit recognition relied on handcrafted feature extraction techniques and classical machine learning classifiers such as Support Vector Machines (SVMs) and K-Nearest Neighbours (KNN). While effective on constrained datasets, these methods exhibited limited scalability and reduced performance when exposed to diverse real-world handwriting styles.

The emergence of deep learning, and in particular Convolutional Neural Networks (CNNs), has dramatically improved the accuracy and robustness of image recognition systems. CNNs are capable of automatically learning hierarchical spatial features directly from raw pixel data, eliminating the need for manual feature engineering. This property makes them especially well-suited for handwritten digit recognition tasks.

This project presents the end-to-end design and implementation of a Handwritten Digit Recognition System using a CNN model trained on the MNIST dataset. The system is deployed as a Flask-based web application, enabling users to upload a handwritten digit image and receive a real-time classification result. The project integrates deep learning, image processing, and web development into a cohesive, functional system.

1.2 Literature Review

Several research contributions have advanced the state of handwritten digit recognition. LeCun et al. [1] introduced the LeNet architecture and demonstrated the effectiveness of convolutional networks for document recognition using the MNIST benchmark, establishing the foundational principles of CNN-based image classification.

Simard et al. [2] investigated best practices for training CNNs on visual document analysis tasks, introducing techniques such as elastic deformation for data augmentation that

significantly improved generalisation. Their experiments showed that CNNs substantially outperformed classical methods on the MNIST dataset.

Goodfellow et al. [3] provided a comprehensive theoretical framework for deep learning, covering the architectural components including convolutional layers, pooling operations, dropout regularisation, and activation functions such as ReLU and Softmax, all of which are employed in this project.

Recent developments have also focused on deploying trained models within web-based frameworks, enabling accessible, real-time inference systems. Flask has emerged as a widely adopted lightweight Python web framework suitable for serving machine learning models through REST APIs [4].

1.3 Objectives

The objectives of this project are as follows:

- To understand and implement a Convolutional Neural Network for handwritten digit image classification.
- To train a CNN model on the MNIST dataset and evaluate performance across multiple training configurations.
- To select the optimal model based on test accuracy, test loss, and real-world generalisation.
- To develop an image preprocessing pipeline that converts user-uploaded images to the format expected by the trained model.
- To build a Flask-based web application that integrates the trained CNN model for real-time digit prediction.
- To design an intuitive and responsive HTML/CSS user interface for the web application.

1.4 Significance

Automated handwritten digit recognition reduces manual effort and human error in data entry tasks across banking, postal, and administrative sectors. This project demonstrates how a lightweight, accurate deep learning model can be packaged into a deployable web application, bridging the gap between academic machine learning research and practical software

deployment. It also provides a reproducible template for similar image classification deployment projects.

1.5 Research Design

This project follows an experimental and implementation-based research design. A Convolutional Neural Network (CNN) model is designed, trained, and evaluated using the MNIST dataset. Multiple training configurations (varying epochs) are tested to analyse performance and select the optimal model based on both benchmark accuracy and real-world generalisation.

1.6 Research Design

The primary dataset used in this project is the MNIST dataset, consisting of 70,000 grayscale images of handwritten digits (0–9). The dataset is accessed using the TensorFlow Keras library. Additionally, real-world handwritten digit samples were collected manually for testing and evaluation of model generalisation.

1.7 Chapter Schema

This report is organised into the following chapters:

Chapter 1 presents the introduction, literature review, objectives, and significance of the project.

Chapter 2 describes the requirements specification, including functional and non-functional requirements.

Chapter 3 explains the system design, including the CNN architecture and system diagrams.

Chapter 4 covers implementation details, tools used, and testing procedures.

Chapter 5 discusses the results, system performance, and interface.

Chapter 6 provides conclusions drawn from the project.

Chapter 7 outlines the future scope and possible enhancements.

Chapter 2 - Requirements Specification

2.1 User Characteristics

The system is intended for two primary user categories:

- **General Users:** Individuals with basic computer literacy who wish to test or use the digit recognition capability. They interact with the system through the web browser interface by uploading an image and viewing the prediction result. No technical knowledge is required.
- **Developer/Evaluator:** A technically proficient user who may interact with the Flask backend directly, evaluate model performance, or extend the system. Knowledge of Python, Flask, and TensorFlow is assumed.

2.2 Functional Requirements

The functional requirements of the system are summarised in Table 2.1.

Table 2.1: Functional Requirements

Req. ID	Requirement Description	Priority
REQ-1	The system shall allow the user to upload an image file via the web interface.	High
REQ-2	The system shall convert the uploaded image to grayscale.	High
REQ-3	The system shall resize the image to 28×28 pixels.	High
REQ-4	The system shall invert the image colours to match the MNIST white-on-black format.	High
REQ-5	The system shall normalise pixel values to the range [0.0, 1.0].	High
REQ-6	The system shall reshape the processed image to the tensor shape (1, 28, 28, 1).	High
REQ-7	The system shall pass the preprocessed tensor to the loaded CNN model for inference.	High
REQ-8	The system shall extract the predicted class using argmax on the model output.	High
REQ-9	The system shall display the predicted digit on the web interface.	High

REQ-10	The system shall load the trained CNN model from the saved .h5 file at startup.	High
--------	---	------

2.3 Dependencies

The system has the following software dependencies:

- Python 3.8 or above
- TensorFlow 2.x and Keras (model loading and inference)
- Flask 2.x (web server and routing)
- Flask-CORS (cross-origin request support)
- Pillow / PIL (image reading and preprocessing)
- NumPy (array manipulation and normalisation)

2.4 Performance Requirements

The system shall return a prediction result within 2 seconds of image submission under standard local server conditions. The model shall achieve a minimum test accuracy of 97% on the MNIST test set. The web application shall correctly handle concurrent requests without model reload overhead, as the model is loaded once at startup and retained in memory.

2.5 Hardware Requirements

The system is designed to run on standard consumer-grade hardware. No specialised hardware is required. The minimum hardware configuration is as follows:

- Processor: Intel Core i3 (8th generation or equivalent) or above
- RAM: 4 GB minimum (8 GB recommended for TensorFlow)
- Storage: 500 MB available disk space (for Python environment, libraries, and model file)
- Operating System: Windows 10 / Ubuntu 20.04 / macOS 12 or above
- Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge (for the web interface)

2.6 Constraints & Assumptions

The following constraints and assumptions apply to the system:

- The CNN model input is fixed at 28×28 grayscale pixels. Images of any size will be resized, which may reduce recognition quality for very low-resolution inputs.
- The system assumes that the uploaded image contains a single handwritten digit centred within the image frame.
- Input images are assumed to contain dark digit strokes on a light background; the preprocessing pipeline inverts colours to match the MNIST training format (white digit on black background).
- The system does not implement file type validation; it is assumed that users upload valid image files (PNG or JPG).
- The model is pre-trained prior to deployment; no online retraining capability is provided.
- Internet connectivity is optional for local deployment but required for cloud hosting.

Chapter 3 - Design

3.1 Algorithm

3.1.1 CNN Architecture

The CNN model is designed as a Sequential architecture comprising two convolutional blocks followed by a fully connected classification head. The detailed layer configuration is presented in Table 3.1.

3-Tier Architecture - Handwritten Digit Recognition System

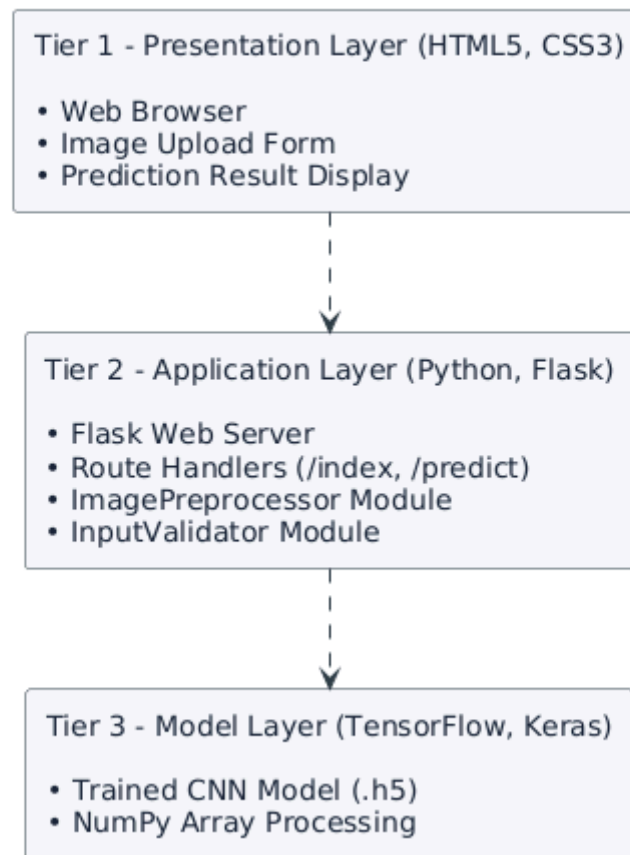


Figure 3.1 : CNN Architecture Block Diagram

Table 3.1: CNN Architecture Layers

Layer	Layer Type	Configuration	Output Shape
1	Conv2D	32 filters, 3×3 kernel, ReLU activation, same padding	(28, 28, 32)
2	MaxPooling2D	Pool size 2×2	(14, 14, 32)
3	Dropout	Rate = 0.25	(14, 14, 32)

4	Conv2D	64 filters, 3×3 kernel, ReLU activation, same padding	(14, 14, 64)
5	MaxPooling2D	Pool size 2×2	(7, 7, 64)
6	Dropout	Rate = 0.25	(7, 7, 64)
7	Flatten	—	(3136)
8	Dense	128 units, ReLU activation	(128)
9	Dropout	Rate = 0.50	(128)
10	Dense (Output)	10 units, Softmax activation	(10)

The first convolutional layer (32 filters) learns low-level spatial features such as edges and curves. The second convolutional layer (64 filters) captures higher-level structural patterns. MaxPooling reduces spatial dimensions, decreasing computational load and introducing spatial invariance. Dropout layers prevent overfitting by randomly deactivating neurons during training. The final Dense layer with Softmax produces a probability distribution over 10 digit classes.

3.1.2 Training Configuration

The model is compiled and trained with the following configuration:

- Optimiser: Adam (adaptive learning rate optimisation)
- Loss Function: Categorical Crossentropy (standard for multi-class classification)
- Batch Size: 128
- Validation Split: 10% of training data(6,000 from train set)
- Dataset Split: Training (60,000 images), Testing(10,000 images)
- Epochs evaluated: 8, 10, 11, 12, 15 (with early stopping analysis)
- Selected final model: 10 epochs (test accuracy ~99.08%, test loss: 0.028)

3.2 System Design

3.2.1 Data Flow Diagrams

DFD Level 0 - Handwritten Digit Recognition System

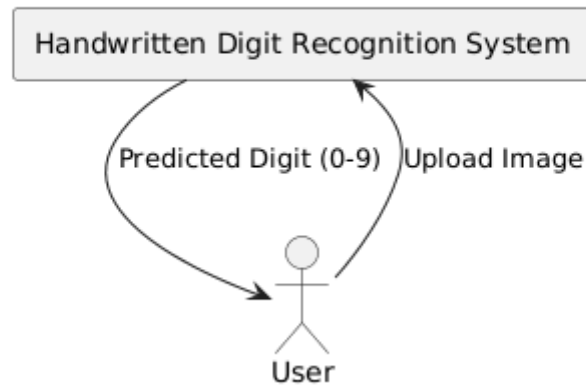


Figure 3.2: Data Flow Diagram – Level 0 (Context Diagram)

The Level 0 DFD represents the system as a single process (P0: Handwritten Digit Recognition System) with one external entity (User). The data flows are:

- User → System: Handwritten digit image (uploaded via web form).
- System → User: Predicted digit label (0–9) displayed on the web interface.

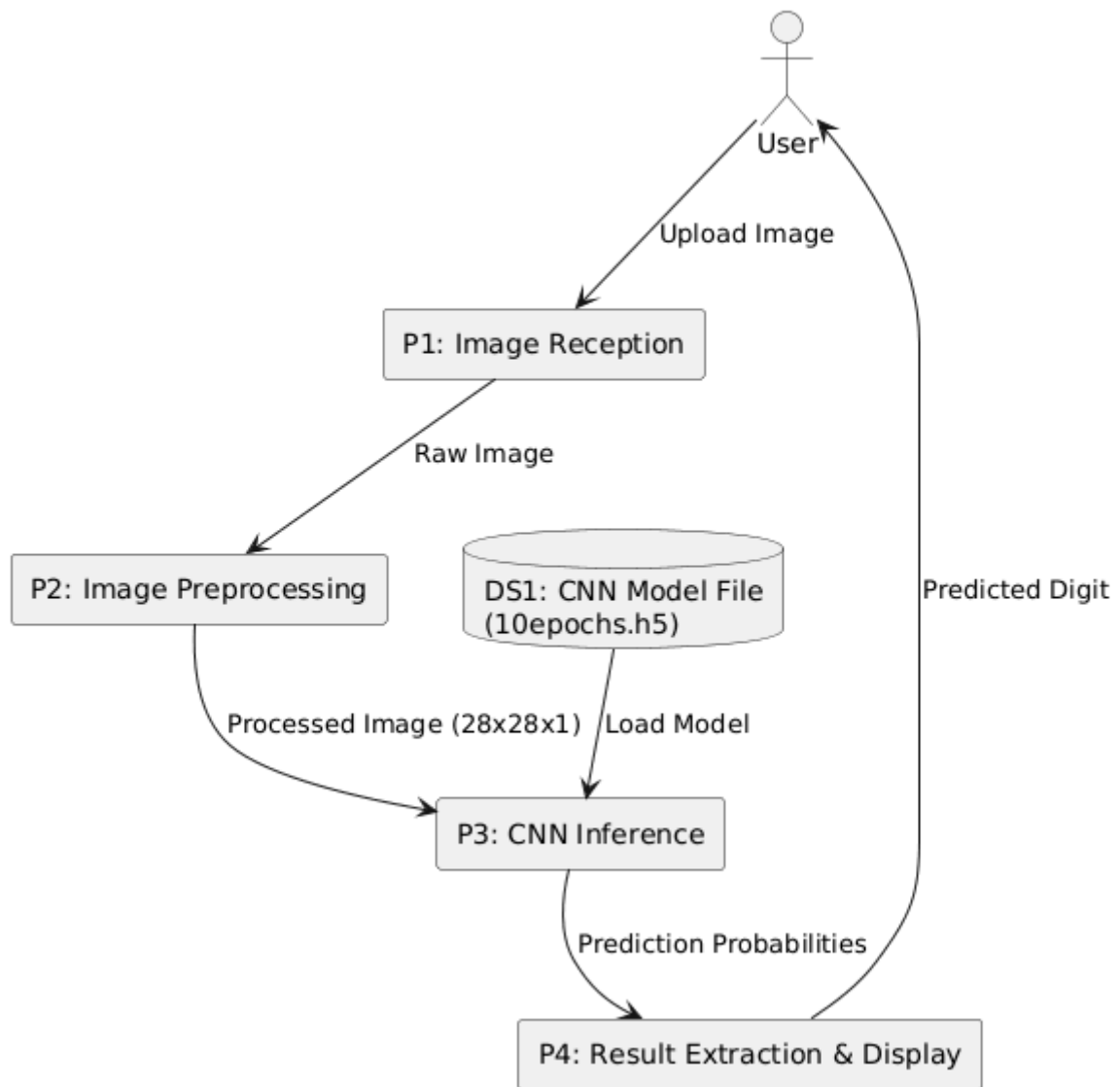
DFD Level 1 - Handwritten Digit Recognition System

Figure 3.3: Data Flow Diagram – Level 1

The Level 1 DFD decomposes the system into the following sub-processes:

- P1 – Image Reception: Receives the uploaded image file from the user via an HTTP POST request to the Flask /index route.
- P2 – Image Preprocessing: Applies grayscale conversion, 28×28 resize, colour inversion, pixel normalisation, and reshaping to produce the model-ready tensor.
- P3 – CNN Inference: Passes the preprocessed tensor to the loaded CNN model and invokes `model.predict()` to obtain a (1, 10) probability array.

- P4 – Result Extraction and Display: Applies `numpy.argmax()` to obtain the predicted digit class and passes the result to the Jinja2 template for display.
- Data Store DS1 – CNN Model File (8epochs.h5 / 10epochs.h5): Persistent model weights read at application startup by P3.

3.2.2 Activity Diagram

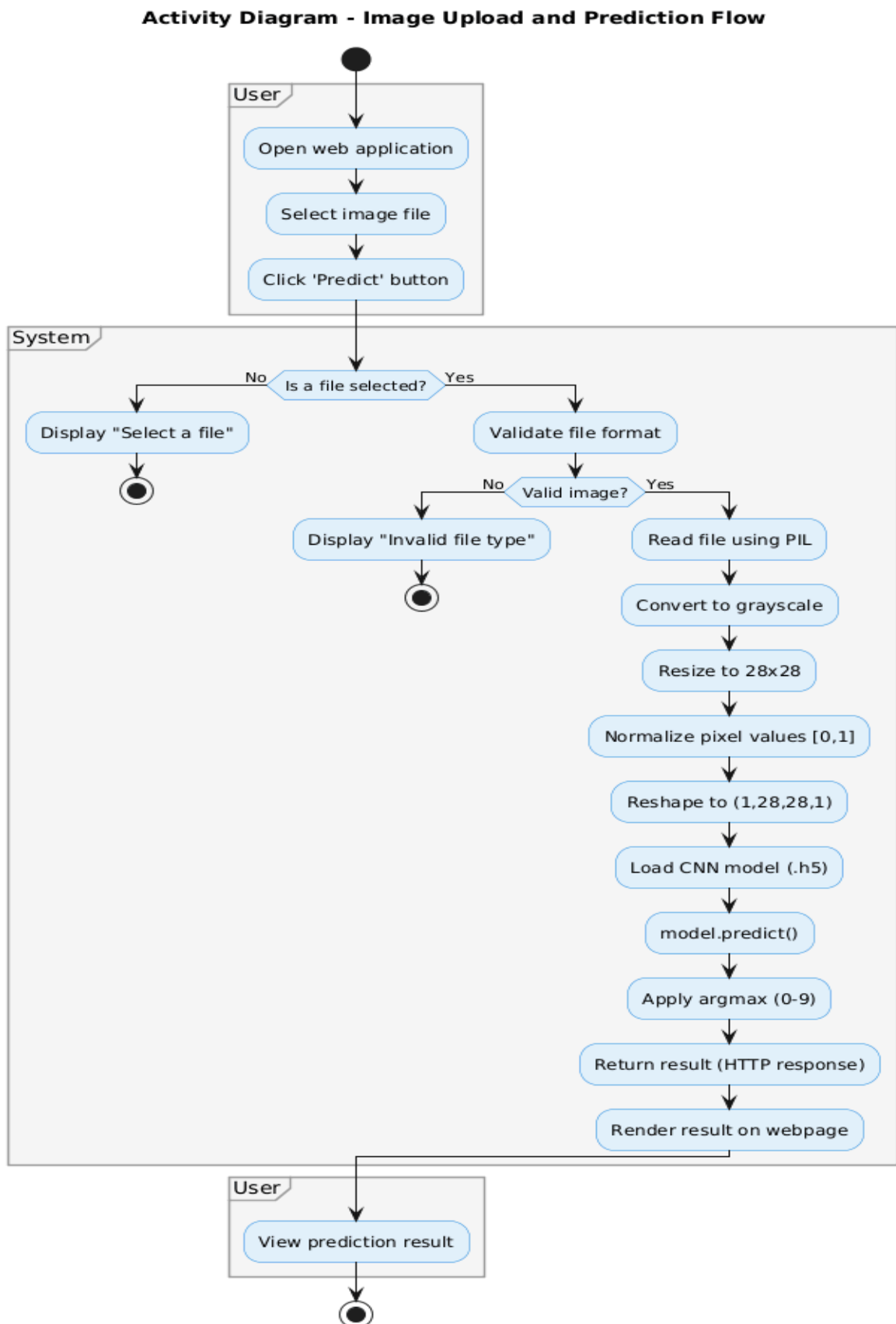


Figure 3.4: Activity Diagram – Digit Prediction Workflow

The activity diagram captures the sequential workflow from image upload to result display, with decision branches for error handling. The flow is as follows:

- Start: User opens the web application in a browser.
- Activity: User selects a handwritten digit image file using the file upload control.
- Activity: User clicks the 'Predict' button, triggering an HTTP POST request to the Flask server.
- Decision: Is a file attached to the request? [No] → Render page with no result (GET behaviour). [Yes] → Proceed.
- Activity: Flask reads the uploaded file from `request.files['image']`.
- Activity: PIL opens the image and passes it to the `preprocess_image()` function.
- Activity: Image is converted to grayscale using `image.convert('L')`.
- Activity: Image is resized to 28×28 pixels using `image.resize((28, 28))`.
- Activity: Colours are inverted using `ImageOps.invert(image)` to match MNIST format.
- Activity: Image is converted to a NumPy array and normalised by dividing by 255.0.
- Activity: Array is reshaped to (1, 28, 28, 1) for CNN input.
- Activity: `model.predict()` is called with the preprocessed tensor.
- Activity: `numpy.argmax()` is applied to extract the predicted digit class.
- Activity: The prediction is passed to `index.html` via `render_template()` and displayed.
- End: User views the predicted digit on the web page.

3.2.3 Sequence Diagram

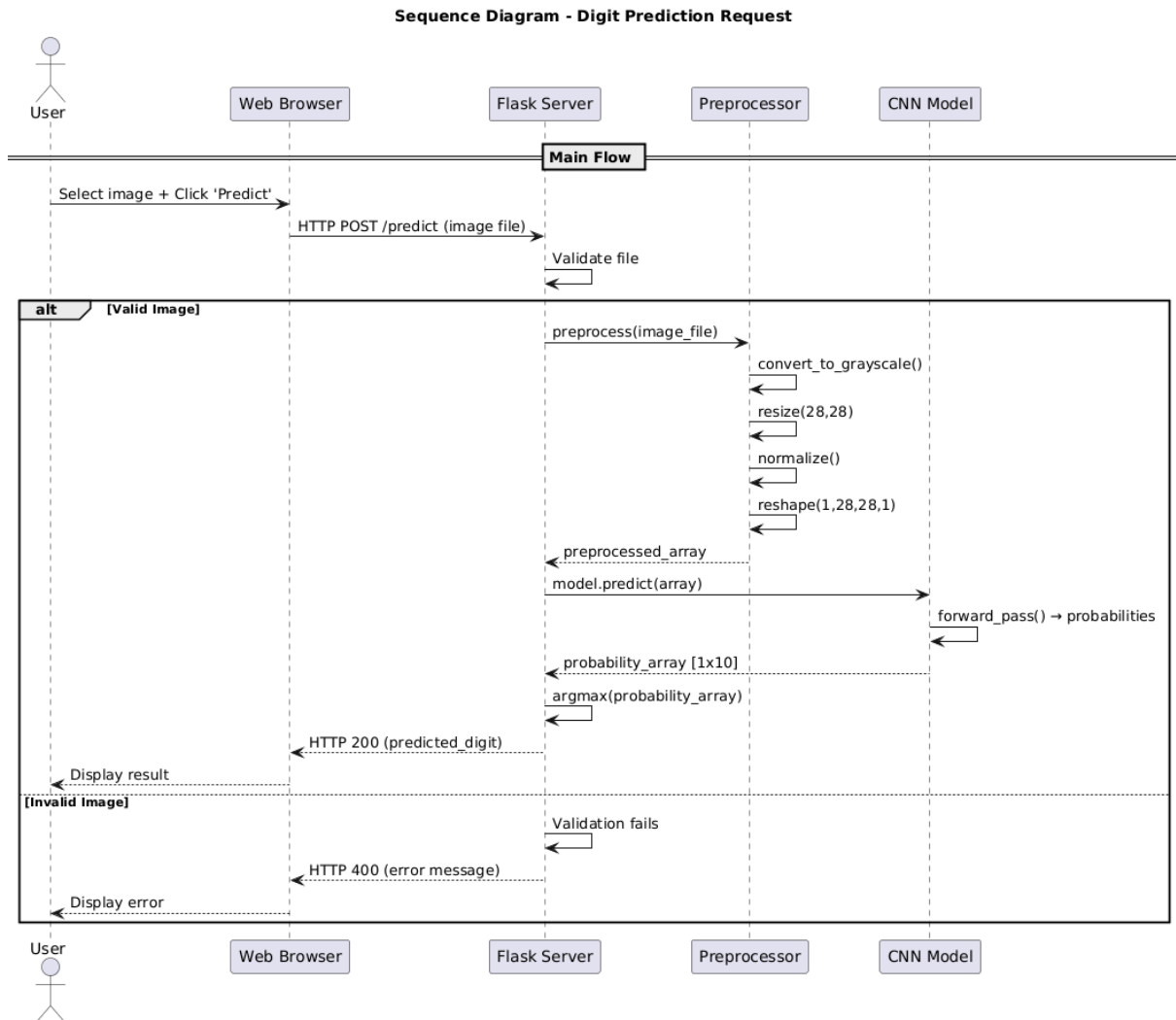


Figure 3.5: Sequence Diagram – Prediction Request Flow

The sequence diagram illustrates the temporal order of interactions between the User, Web Browser, Flask Server, preprocess_image() function, and CNN Model (keras.Model):

- 1. User → Browser: Selects image file and clicks 'Predict'.
- 2. Browser → Flask Server: HTTP POST /index with multipart/form-data payload containing the image file.
- 3. Flask Server: Reads file = request.files['image'] and opens it with Image.open(file).
- 4. Flask Server → preprocess_image(): Calls preprocess_image(image).
- 5. preprocess_image(): Executes convert('L') → resize((28,28)) → ImageOps.invert() → np.array() → / 255.0 → reshape(1,28,28,1).
- 6. preprocess_image() → Flask Server: Returns processed NumPy array of shape (1, 28, 28, 1).

- 7. Flask Server → CNN Model: Calls `model.predict(processed_image)`.
- 8. CNN Model → Flask Server: Returns probability array of shape (1, 10).
- 9. Flask Server: Computes `prediction = np.argmax(preds)`.
- 10. Flask Server → Browser: HTTP 200 response; `render_template('index.html', prediction=prediction)`.
- 11. Browser → User: Renders 'Predicted Digit: X' on the page.

3.2.4 Class Diagram

Class Diagram - Flask-Based Digit Recognition System

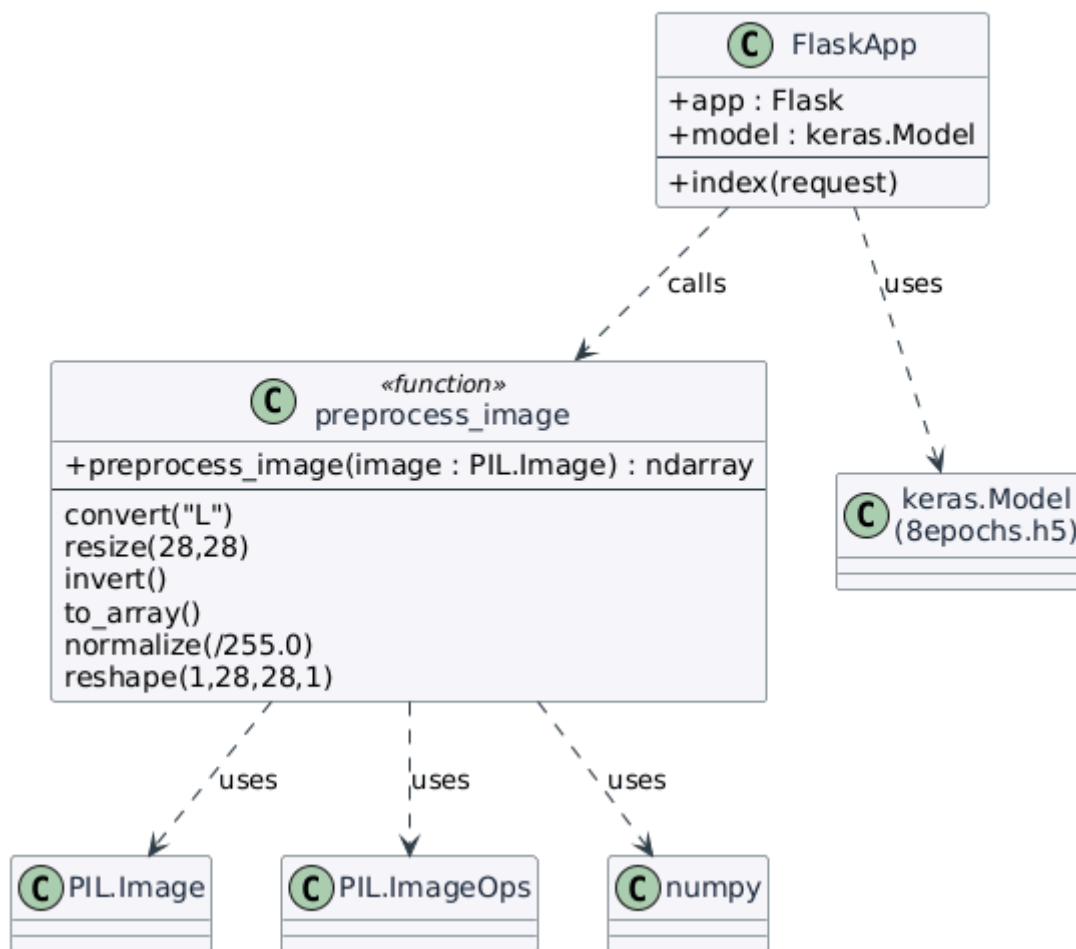


Figure 3.6: Class Diagram – System Components

The system is implemented as a single-module Flask application (`app.py`). The class diagram reflects the actual code structure with two logical components and one external entity:

Module: FlaskApp (`app.py`)

- Module-level attributes: app: Flask, model: keras.Model (loaded from saved .h5 file at startup)
- Route handler: index(request) – handles GET (render upload page) and POST (receive image, call preprocess_image(), call model.predict(), compute argmax, render result)
- Imports: Flask, flask_cors.CORS, tensorflow.keras, numpy, PIL.Image, PIL.ImageOps

Function: preprocess_image(image: PIL.Image) → numpy.ndarray

- Defined in app.py as a standalone utility function; called by the index() route on each POST request.
- Pipeline: convert('L') → resize((28,28)) → ImageOps.invert() → np.array() → / 255.0 → reshape(1,28,28,1)
- Output: ndarray of shape (1, 28, 28, 1), dtype float32, values in [0.0, 1.0]

External Entity: keras.Model (8epochs.h5)

- Loaded via tf.keras.models.load_model() at application startup; stored in module-level global variable model.
- Used via model.predict(array) → returns ndarray of shape (1, 10); argmax applied to extract predicted class.
- Relationship: FlaskApp calls preprocess_image() (intra-module function call); FlaskApp uses keras.Model (module-level dependency).

3.3 Database Design

3.3.1 Logical Database Design

The Handwritten Digit Recognition System does not employ a persistent relational database. All data is processed in memory during each prediction request. However, the following data dictionary (Table 3.2) defines the key data elements managed by the system.

Table 3.2: Data Dictionary

Field Name	Data Type	Description / Constraints
uploaded_file	FileStorage (Flask)	Raw image file received from HTTP POST request. Accepted formats: PNG, JPG, JPEG.

image_grayscale	PIL.Image (mode 'L')	Single-channel grayscale representation of the uploaded image.
image_resized	PIL.Image (28×28)	Image scaled to 28×28 pixels using PIL resize; matches MNIST dimensions.
image_inverted	PIL.Image	Colour-inverted image (white digit on black background) matching MNIST training format.
image_array	numpy.ndarray (float32)	Pixel array after np.array() conversion; shape (28, 28), values in [0.0, 1.0].
model_input	numpy.ndarray (float32)	Final model-ready tensor; shape (1, 28, 28, 1). Batch and channel dimensions added.
probability_output	numpy.ndarray (float32)	Softmax output of CNN; shape (1, 10). Each value represents class probability.
prediction	int (0–9)	Predicted digit class obtained via numpy.argmax(probability_output).

3.3.2 Physical Database Design

As no persistent database is used, normalisation in the traditional relational sense is not applicable. Data normalisation is applied at the input processing level: pixel values are scaled from the integer range [0, 255] to the float range [0.0, 1.0] by dividing by 255.0. This input normalisation ensures numerical stability during inference and aligns with the preprocessing applied during model training on the MNIST dataset.

If the system is extended in future iterations to include prediction history logging, a minimal relational schema adhering to Third Normal Form (3NF) is proposed: PredictionLog(log_id INTEGER PRIMARY KEY, timestamp DATETIME, predicted_digit INTEGER, image_hash VARCHAR(64)). This schema contains no partial or transitive dependencies and satisfies 3NF.

Chapter 4 - Implementation, Testing, and Maintenance

4.1 Languages, IDEs, Tools and Technologies

The following languages, tools, and technologies were used in the implementation of the system:

Programming Language: Python 3.10 – primary language for model training, backend development, and image preprocessing.

Deep Learning Framework: TensorFlow 2.x with Keras API – used for defining the CNN architecture, compiling the model, training, evaluating, and saving the model as an .h5 file.

Web Framework: Flask 2.x – lightweight WSGI micro-framework used to serve the web application, define route handlers, and interface between the frontend and the CNN model.

Image Processing Library: Pillow (PIL) – used for opening uploaded image files, converting to grayscale, resizing to 28×28 pixels, and inverting colours.

Numerical Computing: NumPy – used for converting PIL Image objects to arrays, applying normalisation ($/ 255.0$), and reshaping arrays to model input format.

Cross-Origin Support: Flask-CORS – enables cross-origin HTTP requests, permitting frontend-backend communication across different origins during development.

Frontend Technologies: HTML5 and CSS3 – used for building the user interface. Flask's Jinja2 templating engine renders the prediction result dynamically within index.html.

Development Environment: Visual Studio Code (VS Code) with Python extension; Jupyter Notebook for model training and evaluation.

Dataset: MNIST – accessed via `tensorflow.keras.datasets.mnist`; 70,000 grayscale images of handwritten digits (0–9) at 28×28 pixel resolution.

4.2 Testing Techniques and Test Plans

The system was tested using a combination of model-level evaluation and system-level functional testing.

4.2.1 Model-Level Evaluation

Multiple training configurations were tested to identify the optimal model. The results are presented in Table 4.1.

Table 4.1: Training Configurations and Results

(Epoch-wise Model Performance)

Epochs	MNIST Test Accuracy	MNIST Test Loss	Real-World Accuracy	Observation
8	~97.5%	~0.080	~60%	Underfitting; model has not converged sufficiently.
10	~99.08%	0.028	~80%	Best balance of accuracy and generalisation. Selected as final model.
11	~98.9%	~0.035	~75%	Marginal improvement; slight instability observed.
12	~98.8%	~0.040	~66%	Begins to overfit; real-world accuracy drops.
15	~99.1%	~0.032	~55%	Significant overfitting; poor real-world generalisation.

4.2.2 System-Level Test Cases

Table 4.2: Test Cases and Results

Test Case	Input	Expected Output	Actual Output	Status
TC-1	Clear PNG image of digit '3'	Predicted Digit: 3	Predicted Digit: 3	Pass
TC-2	Clear JPG image of digit '7'	Predicted Digit: 7	Predicted Digit: 7	Pass
TC-3	Digit '0' on white background	Predicted Digit: 0	Predicted Digit: 0	Pass

TC-4	Ambiguous digit '4' with unclear stroke	Predicted Digit: 4	Predicted Digit: 9	Fail
TC-5	Digit written on coloured background	Correct digit	Correct digit	Pass
TC-6	GET request (no file submitted)	Empty result (no crash)	Page renders without error	Pass

The system was tested on over 30 real-world handwritten digit samples and achieved approximately 80–82% accuracy. All 10 digit classes (0–9) were correctly identified in at least one test instance.

4.3 Installation Instructions

To install and run the Handwritten Digit Recognition System, execute the following steps:

1. Install Python 3.8 or above from <https://www.python.org/downloads/>.
2. Clone or download the project directory to your local machine.
3. Open a terminal and navigate to the project root directory.
4. Install the required dependencies by executing: `pip install flask flask-cors tensorflow pillow numpy`
5. Ensure the trained model file (10epochs.h5 or 8epochs.h5) is present in the project root directory.
6. Start the Flask development server by executing: `python app.py`
7. Open a web browser and navigate to `http://localhost:5000` to access the application.

4.4 End User Instructions

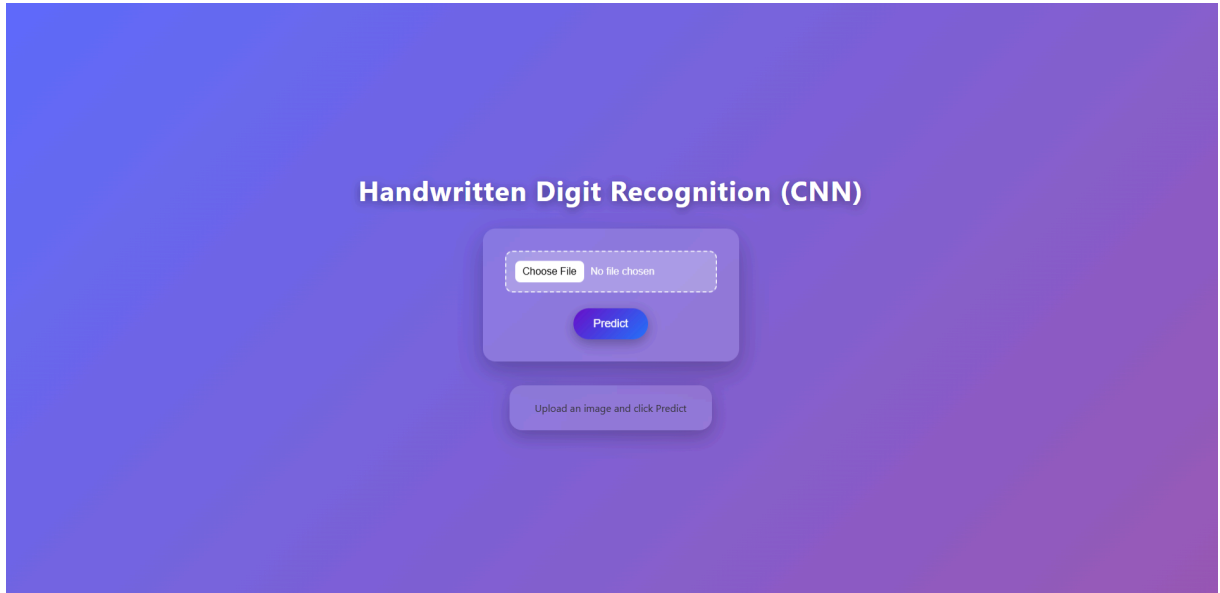
Once the application is running, the user should follow these steps:

8. Open a web browser and navigate to `http://localhost:5000`.
9. On the home page, click the 'Choose File' button to open the file selection dialog.
10. Select a PNG or JPG image file containing a single handwritten digit.
11. Click the 'Predict Digit' button to submit the image.
12. The predicted digit (0–9) will be displayed on the page in the 'Predicted Digit' section.
13. To test another image, click 'Choose File' again and repeat the process.

Chapter 5 - Results and Discussions

5.1 User Interface Representation

5.1.1 Home Page – Upload Interface

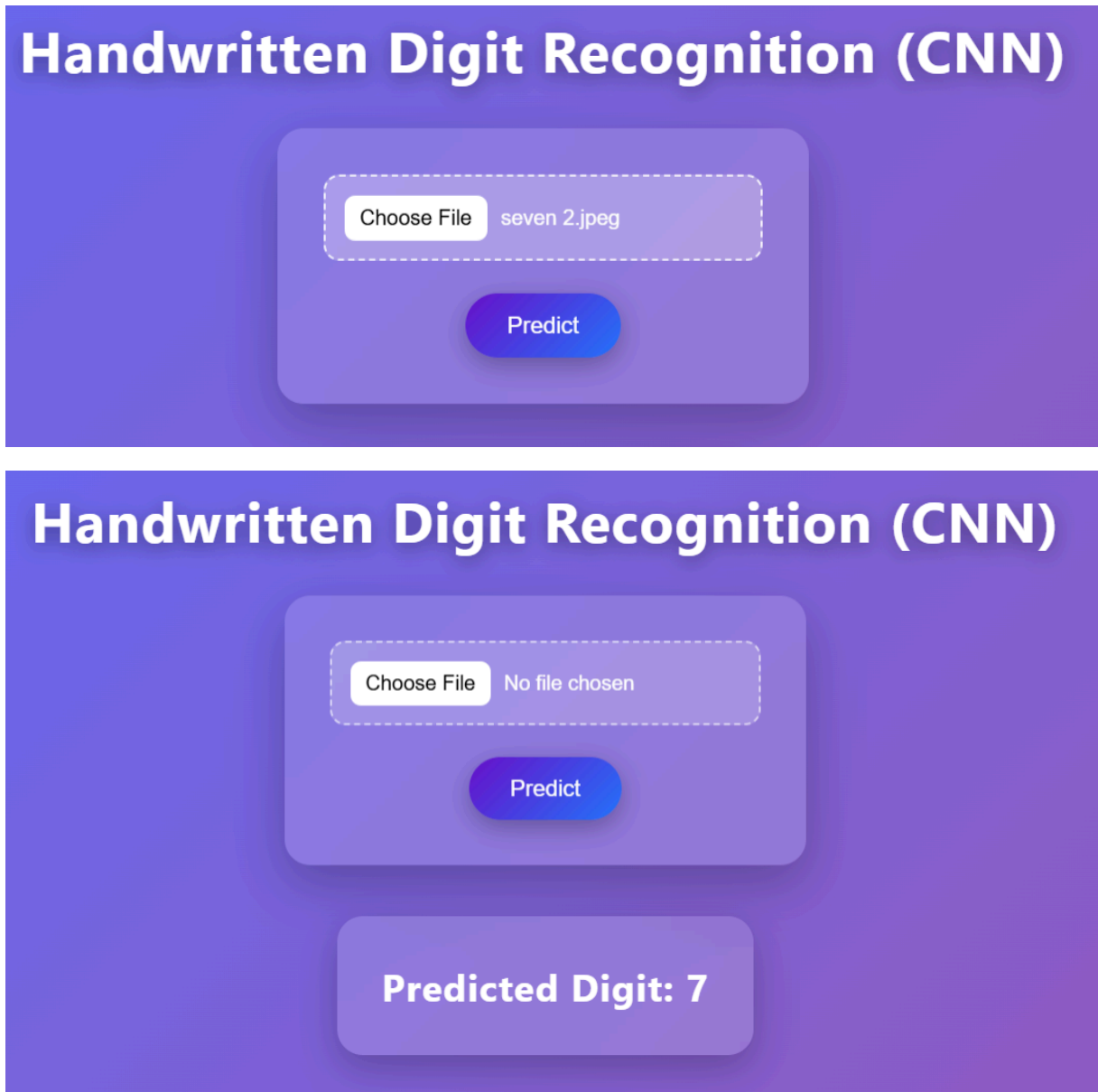


(Figure 5.1) Home Page – Upload Interface

The home page presents a clean, responsive web interface with a gradient background and a card-based central layout. The card contains the following elements:

- Page Title: 'Handwritten Digit Recognition' displayed prominently in large bold text at the top of the card.
- File Upload Control: A styled file input labelled 'Choose File', accepting PNG and JPG image formats. The selected filename is displayed upon selection.
- Predict Button: A prominently styled button labelled 'Predict Digit'. Upon click, the form submits an HTTP POST request to the Flask /index route with the selected image as payload.
- Result Display Area: Initially empty; dynamically populated with the prediction output (e.g., 'Predicted Digit: 7') upon successful model inference. The result is rendered via the Jinja2 template conditional block: `{% if prediction is not none %}`.

5.1.2 Prediction Result Display



(Figure 5.2) Prediction Result Display

After successful image submission and model inference, the same page is reloaded with the result section populated. The predicted digit is displayed in a styled container using large, bold typography for clear readability. The user may immediately upload another image for a new prediction without navigating away.

5.2 Brief Description of System Modules

- **Image Upload Module:** Handles file selection and HTTP POST form submission from the frontend.

- Preprocessing Module (`preprocess_image` function): Executes the full image transformation pipeline – grayscale conversion, resize, colour inversion, normalisation, and reshape.
- Inference Module (CNN model): Performs forward-pass computation on the preprocessed tensor and outputs class probabilities.
- Result Rendering Module (Flask route + Jinja2): Extracts the argmax prediction and passes it to the HTML template for display.

5.3 Snapshots with Brief Description

5.3.1 Snapshot of system

Refer to Figures 5.1 and 5.2 in Section 5.1 for UI snapshots.

The system was validated on 30+ real-world handwritten digit samples. The 10-epoch model correctly identified digits across varying handwriting styles and achieved approximately 80–82% accuracy on real-world inputs. The model demonstrated consistent performance across all digit classes (0–9), with reduced accuracy observed for ambiguous digits such as 1/7 and 4/9 pairs where handwriting style introduced significant visual overlap.

5.3.1 Backend Representation

The backend is implemented entirely in `app.py` using Flask. There is no persistent database. The CNN model weights are stored in the `.h5` (HDF5) binary format on the local filesystem and loaded into memory at application startup via `tf.keras.models.load_model()`. The model occupies approximately 2–5 MB of disk space. All image processing is performed in-memory during each prediction request; no files are saved to the server after processing.

Chapter 6 - Summary and Conclusions

This project successfully designed, developed, and deployed a Handwritten Digit Recognition System using a Convolutional Neural Network trained on the MNIST dataset. The system integrates deep learning, image processing, and web application development into a functional, deployable solution.

The CNN architecture, comprising two convolutional blocks with MaxPooling and Dropout regularisation followed by fully connected dense layers, was trained and evaluated across multiple epoch configurations. The model trained for 10 epochs was identified as the optimal configuration, achieving a MNIST test accuracy of approximately 99.08% (test loss: 0.028) while maintaining approximately 80% accuracy on real-world handwritten digit samples.

The following conclusions are drawn from this investigation:

- CNN-based models are highly effective for handwritten digit recognition, significantly outperforming classical machine learning approaches on the MNIST benchmark.
- Epoch selection has a substantial impact on generalisation: fewer epochs (8) lead to underfitting, while excessive epochs (15) result in overfitting; the optimal is 10 epochs for this architecture and dataset.
- The image preprocessing pipeline, including colour inversion to align with the MNIST training format, is critical for accurate real-world performance.
- Flask provides an effective lightweight framework for deploying machine learning inference models as interactive web applications.
- The system demonstrates that end-to-end ML deployment, from model training to user-facing web interface, is achievable within an academic project scope.

The system correctly identifies all digit classes (0–9) and handles diverse handwriting styles within the constraints of the MNIST-trained model.

Chapter 7 - Future Scope

The following directions are identified for future enhancement of the system:

- **Dataset Expansion:** The current model is trained exclusively on the MNIST dataset, which contains idealised, centred, and uniformly sized digit images. Incorporating additional datasets featuring diverse handwriting styles, noise, and varying orientations will improve real-world generalisation.
- **Data Augmentation:** Applying techniques such as random rotation, elastic deformation, zoom, and shift during training will improve model robustness to handwriting variations without requiring additional training data.
- **Deeper CNN Architectures:** Exploring advanced architectures such as ResNet, VGG, or custom deeper CNNs with batch normalisation may yield higher accuracy on real-world inputs.
- **Improved Preprocessing:** Implementing adaptive thresholding, morphological operations (dilation/erosion), and automatic digit bounding box extraction will reduce sensitivity to image quality and background noise.
- **Multi-Digit Recognition:** Extending the system to detect and recognise sequences of handwritten digits from a single image (e.g., full numbers or postal codes) using object detection techniques.
- **Cloud Deployment:** Deploying the Flask application on a cloud platform (such as AWS, Google Cloud, or Heroku) with a production-grade WSGI server (Gunicorn) to enable public access and scalability.
- **Prediction History Logging:** Implementing a lightweight database (SQLite) to log prediction requests, enabling performance monitoring and retraining data collection.
- **Mobile Application:** Developing a mobile interface using React Native or Flutter that allows users to draw digits directly on a touch screen and receive instant predictions.

Appendix A: Glossary

Term	Definition
CNN (Convolutional Neural Network)	A class of deep learning models designed for processing grid-structured data such as images. CNNs use convolutional layers to automatically extract spatial features through learnable filters.
MNIST Dataset	Modified National Institute of Standards and Technology dataset. A benchmark dataset of 70,000 grayscale images (60,000 training, 10,000 testing) of handwritten digits (0–9) at 28×28 pixel resolution.
TensorFlow	An open-source machine learning framework developed by Google, used in this project for building, training, and deploying the CNN model.
Flask	A lightweight WSGI micro web framework in Python used to build the backend REST API and serve the web application for digit prediction.
ReLU (Rectified Linear Unit)	An activation function defined as $f(x) = \max(0, x)$. Applied in convolutional and dense layers to introduce non-linearity and enable effective deep network training.
Softmax	An activation function applied in the output layer that converts raw model outputs (logits) into a normalised probability distribution over the 10 digit classes, summing to 1.0.
Epoch	One complete pass through the entire training dataset during model training. Multiple epochs allow the model to iteratively refine its learned parameters.
Normalization	The scaling of pixel values from the integer range [0, 255] to the floating-point range [0.0, 1.0] by dividing by 255.0, ensuring consistent input scale for neural network training and inference.
Inference	The process of using a trained machine learning model to generate predictions on new, unseen input data, without any modification to the model's weights.
Overfitting	A condition where a model learns the training data too specifically, capturing noise and irrelevant patterns, resulting in poor performance on unseen test data. Mitigated using Dropout and early stopping.

Bibliography

- [1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, 'Gradient-based learning applied to document recognition,' *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [2] P. Y. Simard, D. Steinkraus, and J. C. Platt, 'Best practices for convolutional neural networks applied to visual document analysis,' in *Proc. Seventh International Conference on Document Analysis and Recognition (ICDAR)*, Edinburgh, UK, pp. 958–963, 2003.
- [3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, pp. 326–366, 2016.
- [4] Pallets Projects, 'Flask – Web Development, one drop at a time,' *Flask Documentation*, version 2.x. [Online]. Available: <https://flask.palletsprojects.com/>. [Accessed: Apr. 2026].
- [5] TensorFlow Authors, 'TensorFlow API Documentation,' version 2.x. [Online]. Available: https://www.tensorflow.org/api_docs. [Accessed: Apr. 2026].
- [6] Y. LeCun and C. Cortes, 'The MNIST Database of Handwritten Digits,' [Online]. Available: <http://yann.lecun.com/exdb/mnist/>. [Accessed: Apr. 2026].
- [7] Pillow Contributors, 'Pillow (PIL Fork) Documentation.' [Online]. Available: <https://pillow.readthedocs.io/>. [Accessed: Apr. 2026].